

基础作业

环境：Python 3.11 + numpy + matplotlib课程资料：robot_localization_system_demo

基础作业

1.1 Demo 复现：里程计漂移与粒子滤波

参考 robot_localization_system_demo ，完成课堂实践流程。
需要完成的流程包括：

- 1. 运行运动学累积误差展示；
- 2. 运行粒子滤波动态演示；
- 3. 查看 outputs/ 目录下生成的结果；
- 4. 查看 metrics.txt 中的最终位姿和误差结果；
- 5. 根据 metrics.txt 计算最终里程计位姿和粒子滤波位姿相对真实位姿的位置误差。
可参考命令：

```
conda activate robot_loc_demo
cd robot_localization_system_demo
python scripts/dead_reckoning_demo.py
python scripts/particle_filter_live_demo.py
```

位置误差计算公式：

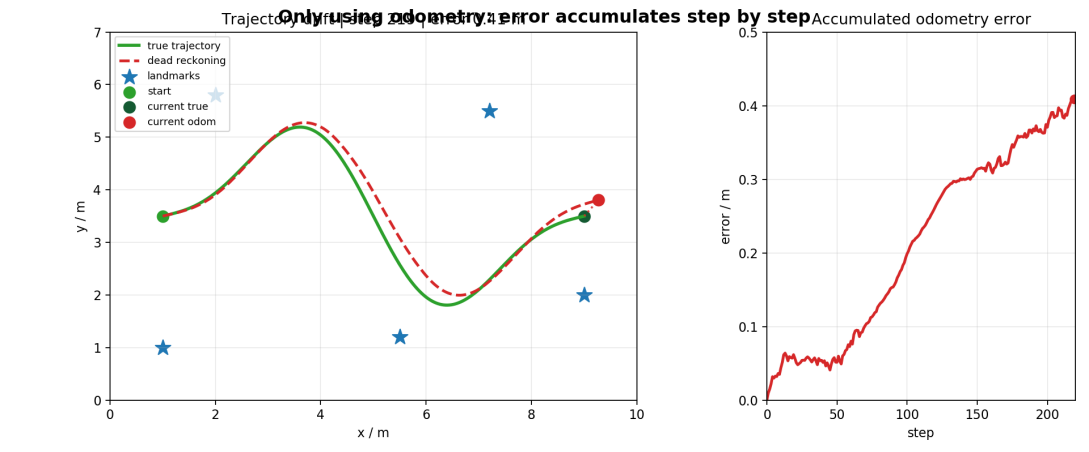
$$error = \sqrt{(x_{true} - x_{est})^2 + (y_{true} - y_{est})^2}$$

提供以下验证材料：

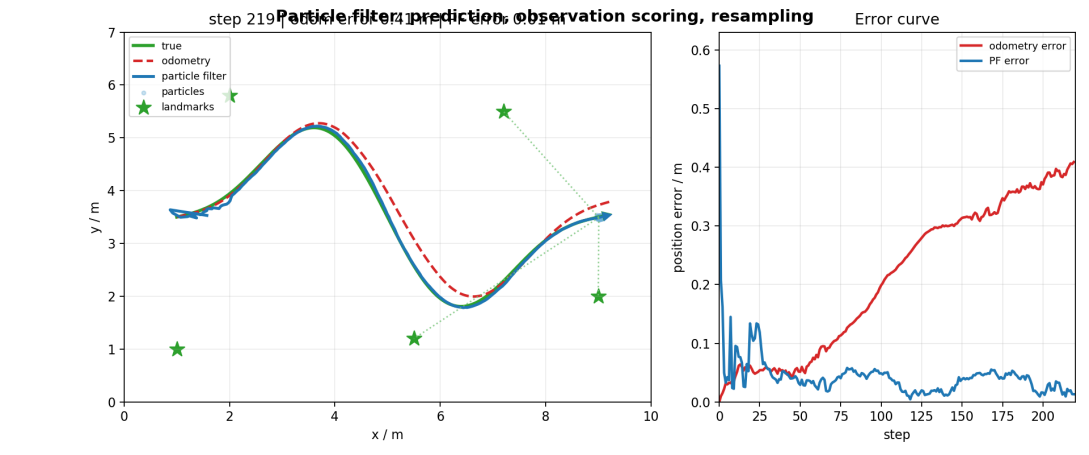
- 运动学累积误差展示截图或 outputs/01_dead_reckoning_drift.png ；
- 粒子滤波结果图 outputs/02_particle_filter_result.png ；
- 误差曲线图 outputs/03_error_curve.png ；
- metrics.txt 的内容截图；
- 最终里程计误差和最终粒子滤波误差的计算过程；
- 简要说明真实轨迹、里程计轨迹、粒子滤波估计轨迹分别表示什么。

1. 运行结果展示

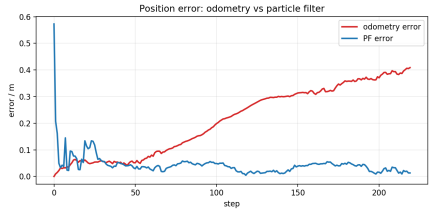
以下是 outputs/ 目录下生成的结果截图：



运动学累积误差展示



粒子滤波结果图



误差曲线图

2. metrics.txt 的内容截图

以下是生成的 metrics.txt 的具体内容：

```
Robot localization demo metrics
Final true pose:      x=9.000, y=3.500, theta=0.232
```

```
Final odometry pose:  x=9.271, y=3.806, theta=0.269
Final PF pose:       x=9.010, y=3.491, theta=0.229
Mean odometry error:  0.211 m
Mean PF error:        0.044 m
Final odometry error: 0.409 m
Final PF error:       0.013 m
```

3. 位置误差计算过程

根据公式 $error = \sqrt{(x_{true} - x_{est})^2 + (y_{true} - y_{est})^2}$ 以及 `metrics.txt` 里的最终数据：

- 真实位姿 (True): $x_{true} = 9.000, y_{true} = 3.500$
1) 最终里程计误差计算 (Odometry Error)
- 里程计位姿 (Odom): $x_{est} = 9.271, y_{est} = 3.806$
- 计算: $error_{odo} = \sqrt{(9.000 - 9.271)^2 + (3.500 - 3.806)^2}$ $error_{odo} = \sqrt{(-0.271)^2 + (-0.306)^2}$ $error_{odo} = \sqrt{0.073441 + 0.093636} = \sqrt{0.167077} \approx 0.409 \text{ m}$
- 结果与 `metrics.txt` 里的 `Final odometry error: 0.409 m` 一致。
2) 最终粒子滤波误差计算 (PF Error)
- 粒子滤波位姿 (PF): $x_{est} = 9.010, y_{est} = 3.491$
- 计算: $error_{pf} = \sqrt{(9.000 - 9.010)^2 + (3.500 - 3.491)^2}$ $error_{pf} = \sqrt{(-0.010)^2 + (0.009)^2}$ $error_{pf} = \sqrt{0.000100 + 0.000081} = \sqrt{0.000181} \approx 0.013 \text{ m}$
- 结果与 `metrics.txt` 里的 `Final PF error: 0.013 m` 一致。可以看到粒子滤波将定位误差显著缩小了。

4. 轨迹简要说明

- 真实轨迹 (True Trajectory): 机器人在仿真环境中实际走过的物理路径。这是评估其他所有估计方法的基准，通常无法被机器人直接测量得知。
- 里程计轨迹 (Odometry Trajectory): 机器人仅仅根据自身车轮编码器或内部控制指令（航位推算）推算出来的运动路径。由于轮子打滑、测量精度等因素，它会产生累积误差，并且运动时间越长，偏离真实轨迹的程度就越大。
- 粒子滤波估计轨迹 (PF Estimated Trajectory): 结合了有噪声的里程计信息与对环境中已知路标（Landmarks）观测信息后，通过粒子滤波算法融合计算得出的路径估计。它利用观测数据不断修正里程计带来的累积偏差，因此能够始终紧紧贴合真实轨迹。

1.2 参数实验：观察定位效果变化

在课堂 Demo 的基础上，自行选择至少 两个参数 进行修改，观察定位效果变化。
可选参数包括：

- 粒子数量 `num_particles`；
- 运动噪声 `motion_noise`；
- 观测噪声 `observation_noise`；
- 传感器观测范围 `sensor_range`；
- 地图地标数量或地标位置 `landmarks`；
- 初始粒子分布参数 `initialization`。

要求：

- 说明你修改了哪些参数；
- 给出修改前后的关键参数值；
- 分别运行粒子滤波演示；
- 对比修改前后的轨迹图、误差曲线和 `metrics.txt`；
- 分析参数变化为什么会影响定位效果。

说明：参数组合可自行选择。请通过对比实验说明这些参数会怎样影响运动预测、观测纠偏、地图先验和状态估计。

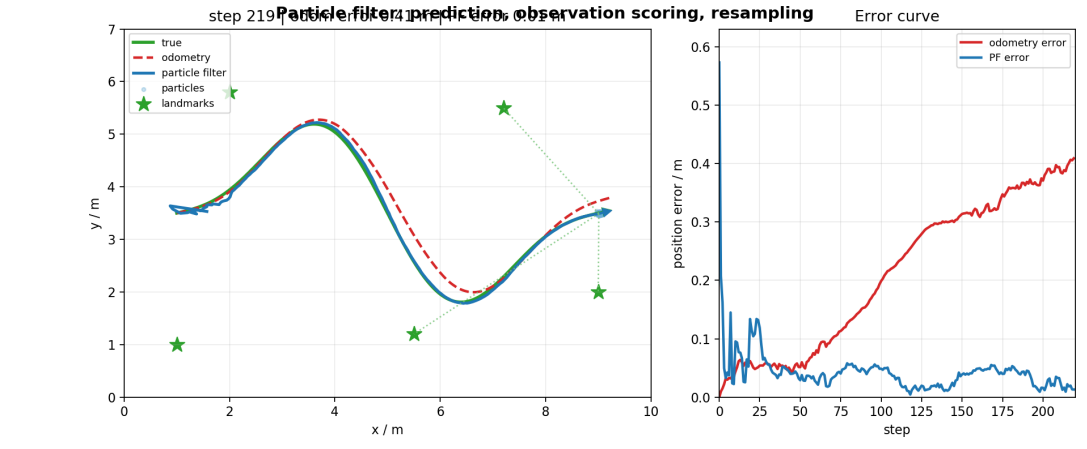
选择了粒子数量 (`num_particles`) 和 传感器观测范围 (`sensor_range`) 两个参数进行同步修改，以此来观察在恶劣条件（算力有限 + 传感器视野受限）下的定位表现。

1. 修改的参数及关键参数值对比

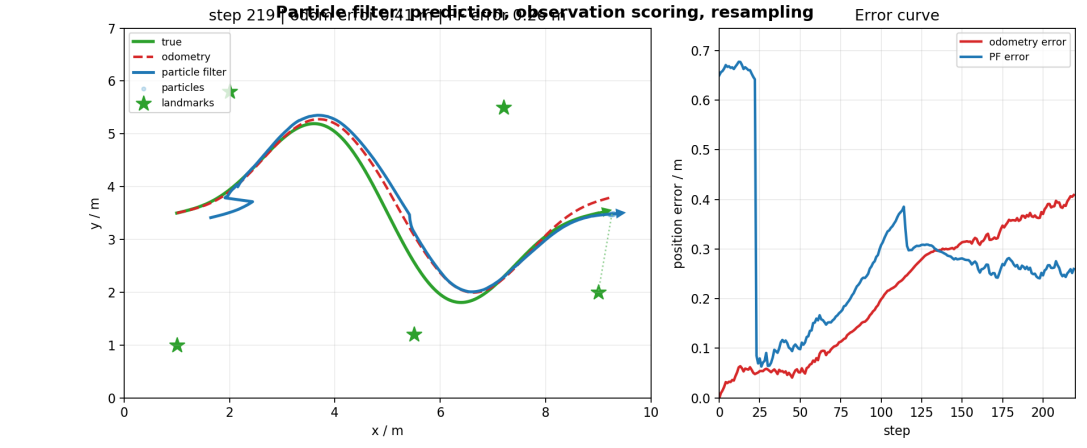
修改的文件为 `config/demo_config.json`：

- 参数 1: 粒子数量 (`num_particles`)
 - 修改前: `400`
 - 修改后: `30`
- 参数 2: 传感器观测范围 (`sensor_range`)
 - 修改前: `4.2`
 - 修改后: `2.0`

2. 轨迹图对比

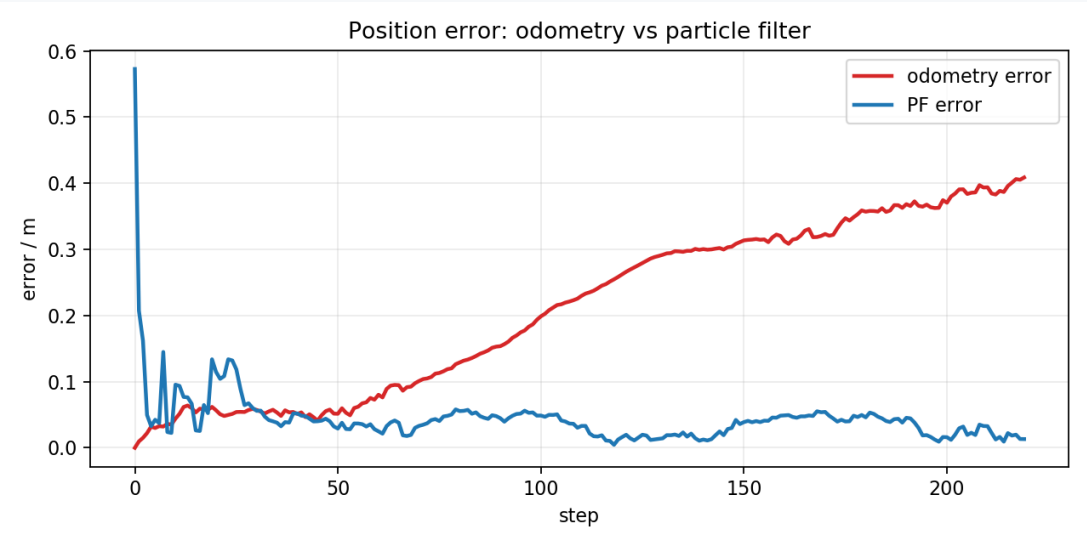


【修改前：粒子多(400)，视野广(4.2)】
(说明：蓝色虚线完美贴合绿色实线，粒子滤波估计位姿非常精确)

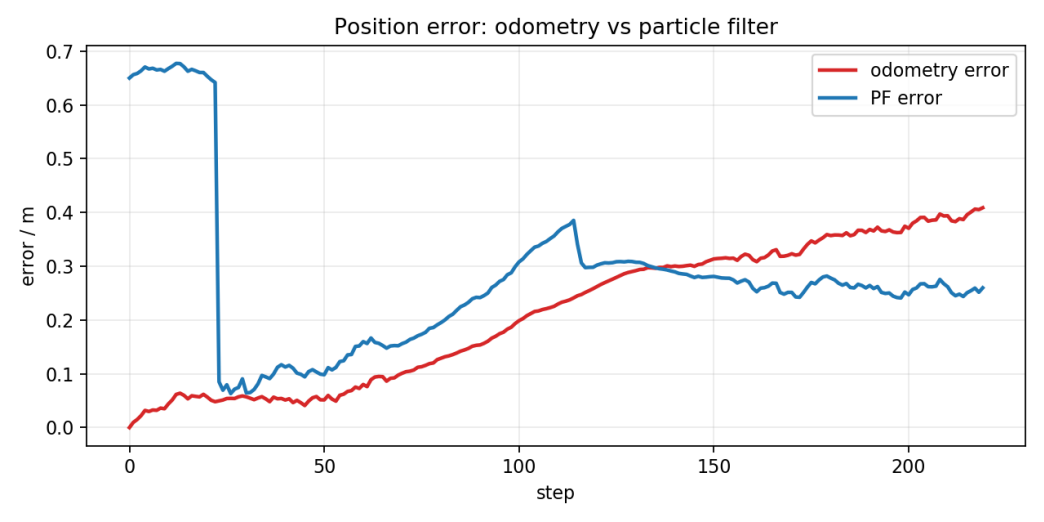


【修改后：粒子极少(30)，视野窄(2.0)】
(说明：蓝色虚线在部分路段明显偏离绿色实线，有些地方甚至紧贴红色的里程计误差轨迹，粒子分布非常稀疏且跳跃)

3. 误差曲线图对比



【修改前】(说明：蓝线 (PF误差) 始终保持在很低的水平)



【修改后】(说明：蓝线出现了数次巨大的峰值，特别是在中间和后半段，说明定位发生了严重漂移或短暂丢失)

4. metrics.txt 结果对比

【修改前】

Final odometry error:	0.409 m	
Final PF error:	0.013 m	(极其精确)
Mean PF error:	0.044 m	

【修改后】

Final odometry error:	0.409 m	
Final PF error:	0.260 m	(误差剧增)
Mean PF error:	0.278 m	(整体平均误差飙升了6倍多)

5. 参数变化影响定位效果的分析

修改的这两个参数组合，对于对机器人的定位系统有较大影响：

1. 减少传感器观测范围 (**sensor_range** 4.2 -> 2.0)：影响“观测纠偏”和“地图先验”
- 分析：在原先 4.2 的视野下，机器人几乎在地图的任何位置都能同时看到至少 1 到 2 个路标。当视野缩小到 2.0 后，机器人在地图中间的广阔区域（特别是 **x=3** 到 **x=5** 的区间）会陷入“盲区”，完全看不到任何路标。

结果：一旦失去地标观测，观测模型无法计算权重（无法纠偏），粒子滤波就会退化成纯粹的里程计推算。这也就是为什么在修改后的轨迹图中，中间路段的蓝色 PF 轨迹和红色里程计轨迹几乎重合，误差急剧上升。
2. 大幅减少粒子数量 (**num_particles** 400 -> 30)：影响“状态估计”和“运动预测”
- 分析：粒子滤波通过一群随机粒子的分布来代表机器人的状态概率密度。400 个粒子足以在状态空间中覆盖真实位姿周围的高概率区域。而当粒子数剧减到 30 时，系统会发生“粒子匮乏 (Particle Deprivation)”。

结果：

运动预测更脆弱：在机器人盲行（无地标）阶段，30 个粒子迅速发散，无法有效维持对各种可能状态的假设。

状态估计易跳变：当机器人重新看到路标时，由于总粒子数太少，可能在真实位姿附近根本没有存活的粒子。重采样算法只能把权重赋予给“勉强靠近”的粒子，导致定位结果（粒子均值）发生剧烈跳变（如修改后的误差曲线中的巨大尖峰）。
- 总结： 传感器视野决定了纠偏的机会和频率，粒子数量决定了系统对不确定性的表达能力和鲁棒性。这说明在实际工程中，如果传感器性能差，我们就必须增加粒子数量和算力来弥补；如果算力有限（粒子少），我们就需要更可靠长距离的传感器。
- ### 1.3 Aelos 相对运动误差观察
- 使用 Aelos 机器人执行同一个固定动作或移动指令，观察真实机器人运动结果是否稳定。
可选方式：
- 前进若干步；

原地转向；

前进 + 转向 + 再前进；

执行一段自定义动作序列。

要求：
1. 说明你选择的动作或移动指令；

2. 记录机器人初始位置和朝向；

3. 重复执行至少 3 次；

4. 记录每次执行后的实际位置、朝向或大致落点；

5. 比较多次结果是否一致；

6. 分析为什么同一个动作在真实机器人上可能产生不同落点。
说明：本题只要求做现象观察和原因分析，不要求完成 Aelos 的精确定位。可以使用人工测量、地面标记、拍照记录或视频截图作为依据。
- ### 1.4 Aelos 视觉地标辅助定位方案设计
- 结合 Aelos 机器人，设计一个利用视觉地标辅助机器人自定位的方案。
可选地标包括：
- ArUco / AprilTag；

固定二维码；

彩色标志物；

场地边界、球门、角点或其他人工标志。

要求：
1. 说明你选择的视觉地标类型；

2. 说明地标在环境中的已知位置如何获得；

3. 说明 Aelos 如何获取图像并检测地标；

4. 说明地标观测如何用于修正机器人位姿；

5. 画出从相机图像到机器人 pose 修正的流程图；

6. 分析该方案可能受到哪些因素影响，例如遮挡、光照、识别误差、相机安装误差和地标布置不合理等。
说明：本题只需要给出可执行的方案设计，不要求完整实现地标定位算法。请说明这个方案如何利用运动预测、观测纠偏和地图先验帮助 Aelos 修正位姿。
- ### 基础作业提交要求
- 请将基础作业整理为 一份 PDF，包含：
1. 课堂 Demo 复现截图；

2. 运动学漂移图、粒子滤波结果图和误差曲线图；

3. **metrics.txt** 关键输出；

4. 最终定位误差计算过程；

5. 参数实验对比结果；

6. Aelos 相对运动误差观察记录与分析；

7. Aelos 视觉地标辅助定位方案。